

add_taxon

User guide — adding rows to *wlist* via the Python utility

Overview

`add_taxon` is a Python utility that provides a batch-friendly, pre-validating interface to the PostgreSQL stored procedure `wlist_add_row`. It supports CSV and JSON input, all-or-nothing transactions, pre-commit validation against the live database, and a dry-run mode that surfaces server NOTICES without persisting any changes.

The utility complements rather than replaces direct invocation of `wlist_add_row`: the procedure remains the canonical contract and any PostgreSQL client can call it directly. `add_taxon` is the recommended path when adding multiple rows or when pre-commit validation is desirable.

Files

<code>add_taxon.py</code>	Main script and importable module
<code>sample_add_taxon.csv</code>	Template CSV — safe for dry-run testing
<code>sample_add_taxon.json</code>	Template JSON — safe for dry-run testing
<code>README.md</code>	Full technical reference (this document is derived from it)

Quick Start

Always run with `--dry-run` first to validate input and inspect server NOTICES before committing. The dry run executes the full transaction then rolls it back — no data is changed.

Dry-run from CSV (recommended first step)

```
python -m wlist.add_taxon.add_taxon --csv wlist/add_taxon/sample_add_taxon.csv --dry-run

# Alternative if running as a plain script:
python wlist/add_taxon/add_taxon.py --csv wlist/add_taxon/sample_add_taxon.csv --dry-run
```

Commit from CSV

```
python -m wlist.add_taxon.add_taxon --csv /path/to/your_rows.csv
# or
python wlist/add_taxon/add_taxon.py --csv /path/to/your_rows.csv
```

Dry-run from JSON

```
python -m wlist.add_taxon.add_taxon --json wlist/add_taxon/sample_add_taxon.json --dry-run
```

Commit from JSON

```
python -m wlist.add_taxon.add_taxon --json /path/to/your_rows.json
```

Command-Line Arguments

Parameter	Type	Description
--csv PATH	str	Path to a CSV file. Headers may be friendly names or exact procedure parameter names (see Input headers below).
--json PATH	str	Path to a JSON file containing one object or an array of objects. Keys may be friendly names or exact parameter names.
--dry-run	flag	Execute and display server NOTICES, then roll back. No changes are persisted.

Environment Variables

Connection details are read by `package_wlist.get_db_connection()`. Set these in your environment before running the utility:

Parameter	Type	Description
DCORE_HOST	str	Database host. Default: localhost
DCORE_DB	str	Database name. Default: dcoredb
DCORE_USER	str	Database user. Default: current OS user
DCORE_PASSWORD	str	Database password.
DCORE_PORT	int	Database port. Default: 5432

Input Headers

CSV and JSON input may use either **friendly names** (recommended) or the exact stored-procedure parameter names. The mapping is applied automatically. Missing fields are treated as NULL.

Friendly names

taxon_sort_target	is_ultrataxon	taxon_level
sp_id	taxon_id	taxon_name
taxon_scientific_name		family_name
family_scientific_name		order (alias: t_order)
population	aust_rli_1990	aust_rli_2000
aust_rli_2010	aust_rli	bird_group
supplementary	avibase_id	reference

Exact procedure parameter names

p_taxon_sort_target	p_taxon_id	p_is_ultrataxon
p_taxon_level	p_sp_id	p_taxon_name
p_taxon_scientific_name		p_family_name
p_family_scientific_name		p_t_order
p_population	p_aust_rli_1990	p_aust_rli_2000
p_aust_rli_2010	p_aust_rli	p_bird_group
p_supplementary	p_avibase_id	p_reference

Integer-like fields validated and cast: p_taxon_sort_target, p_is_ultrataxon, p_sp_id, p_aust_rli_1990, p_aust_rli_2000, p_aust_rli_2010, p_aust_rli, p_supplementary.

Legacy alias supported: bird_sub_group is accepted and mapped to bird_group.

CSV Format

Encoding	UTF-8 without BOM. Excel's "CSV UTF-8" option may insert a BOM — prefer plain "CSV (comma delimited)" or remove the BOM before loading.
Delimiter	Comma (.).
Quoting	Double quotes (") for fields containing commas, double quotes, or newlines. Escape a literal double quote by doubling it: """"."
Header row	Required as the first row. Headers must exactly match friendly names or p_* parameter names (case/space insensitive mapping applied).
Whitespace	Leading/trailing spaces trimmed. Empty cells treated as NULL.
Numbers	Plain digits only — no thousands separators or units.
Line endings	LF or CRLF both accepted.

Validation and Behaviour

- Required fields: p_taxon_sort_target and p_is_ultrataxon (0 or 1).
- If p_is_ultrataxon ≠ 1, then p_taxon_id must be provided.
- p_taxon_sort_target must already exist in wlist (safety check to prevent large unintended shifts).
- If p_taxon_id is provided, it must not already exist in wlist (duplicate check).
- If p_sp_id is not provided, the procedure auto-assigns the next available value in the range 5001–7999.

- If `p_taxon_id` is not provided for ultrataxons, the procedure generates `u||sp_id`.
- All rows in a batch are processed within a single transaction. Any failure rolls back all rows (all-or-nothing behaviour).

Examples

CSV — minimal ultrataxon (auto `sp_id` and `taxon_id`)

```
taxon_sort_target,is_ultrataxon,taxon_level,taxon_name,taxon_scientific_name,family_name,family_scientific_name,order,
1729,1,ssp,Common Redpoll (ssp),Acanthis flammea cabaret,Old World Finches,Fringillidae,Passeriformes,Introduced
```

CSV — full field specification (non-ultrataxon)

```
taxon_sort_target,is_ultrataxon,taxon_level,sp_id,taxon_id,taxon_name,taxon_scientific_name,\
family_name,family_scientific_name,order,population,aust_rli_1990,aust_rli_2000,\
aust_rli_2010,aust_rli,bird_group,supplementary,avibase_id,reference
2000,0,species,5123,u5123,Some Bird,Genus species,Family,FamilySci,Order,Native,\
1990,2000,2010,2020,Subgroup,0,ABCD-1234,Ref text
```

JSON — minimal ultrataxon

```
[
  {
    "taxon_sort_target": 1729,
    "is_ultrataxon": 1,
    "taxon_level": "ssp",
    "taxon_name": "Common Redpoll (ssp)",
    "taxon_scientific_name": "Acanthis flammea cabaret",
    "family_name": "Old World Finches",
    "family_scientific_name": "Fringillidae",
    "order": "Passeriformes",
    "population": "Introduced"
  }
]
```

JSON — full field specification

```
[
  {
    "taxon_sort_target": 2000,
    "is_ultrataxon": 0,
    "taxon_level": "species",
    "sp_id": 5123,
    "taxon_id": "u5123",
    "taxon_name": "Some Bird",
    "taxon_scientific_name": "Genus species",
    "family_name": "Family",
    "family_scientific_name": "FamilySci",
    "order": "Order",
    "population": "Native",
    "aust_rli_1990": 1990,
    "aust_rli_2000": 2000,
    "aust_rli_2010": 2010,
    "aust_rli": 2020,
    "bird_group": "Subgroup",
    "supplementary": 0,
    "avibase_id": "ABCD-1234",
    "reference": "Ref text"
  }
]
```

Python API

The module can be imported directly. The primary entry points are `add_wlist_rows_psycpg2()` and `load_rows_from_csv()`.

From CSV

```
from wlist.add_taxon.add_taxon import add_wlist_rows_psycpg2, load_rows_from_csv

rows = list(load_rows_from_csv('wlist/add_taxon/sample_add_taxon.csv'))
result = add_wlist_rows_psycpg2(rows, dry_run=True)
print(result)
```

From JSON file

```
import json
from wlist.add_taxon.add_taxon import add_wlist_rows_psycpg2

with open('wlist/add_taxon/sample_add_taxon.json', 'r', encoding='utf-8') as f:
    rows = json.load(f)
result = add_wlist_rows_psycpg2(rows, dry_run=True)
print(result)
```

From in-memory dictionaries

```
rows = [
    {
        "taxon_sort_target": 1729,
        "is_ultrataxon": 1,
        "taxon_level": "ssp",
        "taxon_name": "Common Redpoll (ssp)",
        "taxon_scientific_name": "Acanthis flammea cabaret",
        "family_name": "Old World Finches",
        "family_scientific_name": "Fringillidae",
        "order": "Passeriformes",
        "population": "Introduced",
    }
]
result = add_wlist_rows_psycpg2(rows, dry_run=False)
print(result)
```

Troubleshooting

Connection errors (*psycpg2.OperationalError*)

Verify that DCORE_HOST, DCORE_DB, DCORE_USER, DCORE_PASSWORD, and DCORE_PORT are set correctly and the database is reachable.

Procedure errors (e.g. *missing p_taxon_id*)

Run with `--dry-run` to see server NOTICES and error messages without committing.

Header issues

Verify that CSV headers exactly match the friendly or `p_*` parameter names listed in Input headers. Check for invisible characters or BOM.

Procedure signature mismatch

If you see "procedure wlist_add_row(...) does not exist", ensure the database has the current procedure signature (uses `bird_group`, not the legacy `bird_sub_group`).

Large unexpected shifts

The tool validates that `p_taxon_sort_target` exists in `wlist` before proceeding. If this check fails, confirm the target sort value using a `SELECT` query.

Notes

`add_taxon` modifies `taxon_sort` ordering in `wlist` by design — it shifts values for all rows below the insertion point. Prefer running during a quiet window to avoid conflicting edits.

The sample CSV and JSON files (`sample_add_taxon.csv` / `sample_add_taxon.json`) are provided as authoring templates. They are safe by default and intended for dry-run testing. Do not hard-code them as default input paths in automated workflows.

Deletion of rows is handled by direct calls to the `wlist_delete_row` procedure. No Python wrapper for deletion is currently provided.